# Design and Integration of a TinyML Systolic Accelerator with PicoRV32 (EdgeMATX)

Major Project Report


**Submitted by:**

Nishchay Pallav 221EC233

Mohammad Omar Sulemani 221EC230

Md Atib Kaif 221EC129

Under the Guidance of:

**Dr Rathamala Rao**


March , 2025


Department of Electronics and Communication Engineering

National Institute of Technology Karnataka Surathkal—575025

# Abstract

This report presents the current progress of the EdgeMATX major project. The core objective is to accelerate matrix multiplication workloads by integrating a custom 4x4 Q5.10 fixed-point hardware accelerator with a RISC-V CPU (PicoRV32) using the PCPI custom instruction interface.

At the current stage, the project has achieved complete simulation-stage functionality across standalone accelerator verification, CPU-accelerator integration, firmware-driven execution, protocol handoff validation, and measured cycle comparisons against software baselines.

Key outcomes include successful regression and handoff tests, reusable automation scripts for evaluation, and measured speedups of 38.8262x (vs software without MUL) and 11.8499x (vs software with MUL) for the validated benchmark case.

For dense data profiles, software no-MUL comparison reaches 50x-class speedup (validated up to 53.8574x in current live profile).

The remaining major milestone is FPGA board deployment and hardware measurement closure (timing, resource, and power evidence).

# Contents

**Abstract** **i**

**1 Introduction** **1**
1.1 Background and Motivation . . . . . 1
1.2 Problem Statement . . . . . 2
1.3 Proposed Approach . . . . . 3
1.4 Project Scope and Current Stage . . . . . 4

**2 Related Work** **5**
2.1 Introduction . . . . . 5
2.2 Systolic Array Architectures for Matrix Multiplication . . . . . 5
2.2.1 FPGA Systolic Array Implementations . . . . . 6
2.2.2 Systolic Arrays in Application-Specific Contexts . . . . . 6
2.3 RISC-V Custom Instructions and CPU–Accelerator Co-design . . . . . 7
2.3.1 The PicoRV32 PCPI Interface . . . . . 7
2.3.2 RISC-V ISA Extensions for Compute-Intensive Workloads . . . . 7
2.4 Fixed-Point Arithmetic and Numerical Representation . . . . . 8
2.4.1 Q-Format Arithmetic . . . . . 9
2.4.2 Fixed-Point Design for FPGAs . . . . . 9
2.5 Application Context: Matrix Acceleration for Compute-Intensive Workloads 10
2.5.1 Motivation: Why Matrix Acceleration Matters . . . . . 10
2.5.2 Related Hardware Accelerator Designs . . . . . 10

**3 System Architecture and Implementation** **12**
3.1 Overall System Architecture . . . . . 12
3.2 Matrix Accelerator Architecture . . . . . 13
3.3 Systolic Array Architecture . . . . . 14
3.4 Processing Element Design . . . . . 15
3.4.1 Processing Element Code Snippet . . . . . 15
3.5 Issue Logic . . . . . 16
3.6 CPU–Accelerator Integration . . . . . 16
3.7 Execution Workflow . . . . . 17
3.8 Memory Mapping . . . . . 18
3.9 Fixed-Point Arithmetic Contract . . . . . 18

**4 Verification Methodology and Results** **19**
4.1 Overview of Verification Approach . . . 19
4.2 Verification Infrastructure and Tools . . . 20
4.3 Layer 1 — Standalone Accelerator Verification . . . 20
4.3.1 Test Configuration . . . 21
4.3.2 What Is Exercised . . . 21
4.3.3 Test Results by Category . . . 21
4.3.4 Simulation Waveform . . . 21
4.4 Layer 2 — System-Level Integration Verification . . . 22
4.4.1 Custom Instruction Encoding . . . 22
4.4.2 PCPI Wrapper Integration Logic . . . 23
4.4.3 Smoke Test (ASM and C Firmware) . . . 23
4.4.4 Integration Waveform . . . 24
4.4.5 8-Case Regression Suite . . . 24
4.4.6 Professor Demonstration Cases . . . 25
4.5 Layer 3 — Protocol Correctness Verification (Handoff Test) . . . 25
4.5.1 PCPI Interface Signals . . . 26
4.5.2 Handoff Test Results . . . 26
4.6 Layer 4 — Performance Benchmarking . . . 27
4.6.1 Measurement Basis . . . 27
4.6.2 Benchmark Case: `identity_x_sequence` . . . 28
4.6.3 Derived Speedups . . . 28
4.6.4 Dense Data Profile (`live_real_input.json`) . . . 29
4.6.5 Why the Acceleration Is Effective . . . 30
4.6.6 Benchmark Fairness and Standardization . . . 31
4.7 Consolidated Verification Summary . . . 32
4.8 Current Limitations and Pending Measurements . . . 32

**5 Conclusion** **34**

# List of Figures


1.1 Matrix multiplication as a fundamental computational primitive in modern computing systems. [1] . . . . . 1
1.2 High-level architecture of the RISC-V system with a matrix multiplication coprocessor. [3] . . . . . 3

2.1 Canonical $4 \times 4$ systolic array dataflow. Data from matrix $A$ streams across rows (red arrows) and data from matrix $B$ streams down columns (green arrows); each PE performs one MAC per cycle and forwards its inputs to its neighbours. This is the topology employed in the EdgeMATX engine. (Adapted from [5].) . . . . . 6
2.2 PCPI handshake timing diagram. The CPU stalls (`pcpi_wait` high) from the cycle it issues the custom instruction until the accelerator asserts `pcpi_ready` and places the result on `pcpi_rd`. This is exactly the protocol implemented in EdgeMATX. (Adapted from [9].) . . . . . 7
2.3 Q5.10 fixed-point format (16-bit, two's complement). Bit 15 is the sign bit; bits 14–10 carry the integer part; bits 9–0 carry the fractional part. Range: $-32.0$ to $+31.999$; resolution: $1/1024 \approx 0.001$. (Adapted from [14].) 8
2.4 System stack for compute-intensive edge workloads. EdgeMATX targets the hardware accelerator layer (highlighted); no ML model deployment or inference testing is performed within the project scope. (Adapted from [18].) 10

3.1 High-level system architecture of the RISC-V processor with the matrix multiplication coprocessor. [23] . . . . . 13
3.2 Interactive web-based visualizations for system architecture and detailed processor–accelerator execution flow. . . . . 13
3.3 Internal architecture of the matrix multiplication accelerator. . . . . 14
3.4 Internal architecture of the matrix multiplication accelerator. [23] . . . . 15
3.5 Execution flow from custom instruction invocation to result writeback. [3] 17

4.1 Four-layer verification hierarchy for EdgeMATX. . . . . 20
4.2 GTKWave waveform from the standalone accelerator regression. *Control Signals* group (top): `clk`, `rst`, `start`, `busy`, `done`. `cycle_count[7:0]` increments from `00` to `0A` (decimal 10), confirming the data-independent 10-cycle systolic latency. `c_flat[255:0]` transitions to the packed Q5.10 result matrix upon `done` assertion. . . . . 22

4.3 GTKWave waveform from the PCPI system-level integration simulation. `pcpi_valid` asserts when the CPU encounters the custom-0 instruction (`pcpi_insn = 0x5420818b`). `pcpi_wait` immediately stalls the processor while the accelerator computes. `accel_mem_addr` and `accel_mem_wdata` show the wrapper reading matrix operands from memory. `pcpi_ready` asserts on completion, releasing the CPU stall and returning control to the instruction stream. . . . . 24
4.4 GTKWave waveform from the PCPI handoff test, spanning 0–17855 ns. Two distinct custom-instruction transactions are visible (first at $\sim 8\,\mu$s, second at $\sim 17\,\mu$s): `pcpi_valid` asserts when the CPU issues `0x5420818b`; `pcpi_wait` immediately stalls the CPU while `accel_mem_addr` sequences through the matrix operand addresses; `pcpi_ready` pulses to release the stall. The interval between the two transactions shows normal `mem_valid` activity, confirming the CPU resumes instruction execution and no stale signals persist across transactions. . . . . 26
4.5 Simulation cycle counts across the three execution paths (`identity_x_sequence` case). Log-scale Y-axis makes the 673-cycle accelerator bar visible alongside the 26,130-cycle software baseline. . . . . 29
4.6 Speedup achieved by EdgeMATX across the two benchmark profiles. The MUL-baseline speedup (11.85×) is data-independent, while the no-MUL speedup increases with operand density (38.83× to 53.86×) because the shift-and-add software path requires more instructions for larger values. . 30

# List of Tables


3.1 Memory layout used for matrix data exchange between CPU and accelerator. 18

4.1 Verification automation scripts and their roles. . . . . 20
4.2 Standalone accelerator test results by category. . . . . 21
4.3 8-case PCPI integration regression results. . . . . 25
4.4 PCPI interface signals. . . . . 26
4.5 Handoff test protocol counters: expected vs. observed. . . . . 27
4.6 Measured simulation cycle counts for the `identity_x_sequence` benchmark. 28
4.7 Derived speedup values for the `identity_x_sequence` benchmark case. . 28
4.8 Cycle counts for the dense `live_real_input.json` profile. . . . . 29
4.9 Consolidated functional validation status across all verification layers. . . 32
4.10 Performance summary: speedup across both benchmark profiles. . . . . 32

# Chapter 1

# Introduction

## 1.1 Background and Motivation

Modern computing systems increasingly rely on large-scale numerical computations. Among these operations, **matrix multiplication** is one of the most fundamental and frequently used computational kernels. Matrix operations appear in a wide variety of domains including machine learning, signal processing, computer graphics, robotics, scientific simulations, and data analytics. Many algorithms in these domains rely heavily on repeated multiply–accumulate (MAC) operations over vectors and matrices.

For example, machine learning algorithms perform dense linear algebra operations during neural network inference and training. Digital signal processing algorithms frequently rely on convolution and filtering operations, which can be expressed as matrix multiplications. Similarly, many scientific computing tasks involve solving systems of linear equations and performing matrix transformations. As a result, matrix multiplication has become a critical operation whose efficiency directly impacts the performance of many computing workloads.

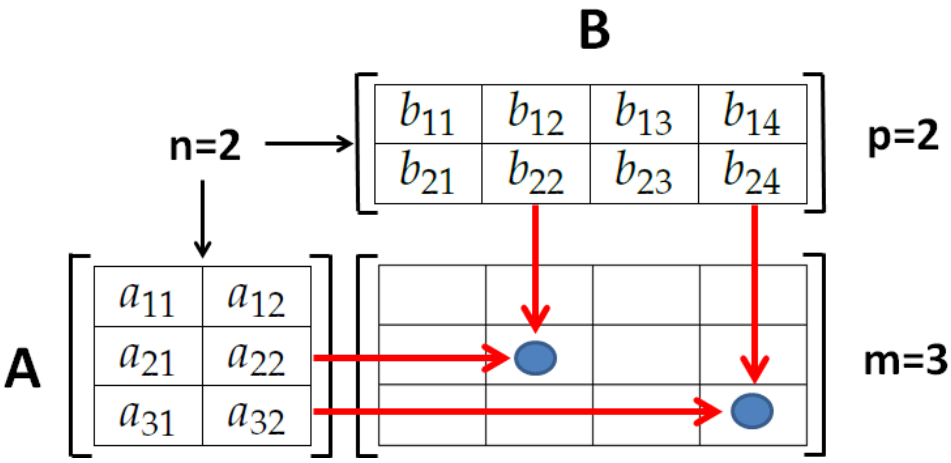


Figure 1.1: Matrix multiplication as a fundamental computational primitive in modern computing systems. [1]

Despite its importance, matrix multiplication can be computationally expensive when

executed on traditional scalar processors. General-purpose CPUs execute instructions sequentially, typically performing one arithmetic operation at a time. When implementing matrix multiplication in software, the computation is usually expressed as nested loops that repeatedly perform multiplication and addition operations. For embedded processors with limited computational resources, this sequential execution leads to a high number of clock cycles for even small matrix sizes.

Lightweight processors such as compact RISC-V cores are particularly affected by this limitation [2]. While these processors provide flexibility and programmability, they are not optimized for performing large numbers of parallel arithmetic operations. As a result, executing matrix multiplication entirely in software can significantly increase execution time and reduce system efficiency.

To overcome these limitations, many modern computing architectures incorporate **hardware accelerators**. Hardware accelerators are specialized computation units designed to perform specific tasks more efficiently than a general-purpose processor. By implementing critical computational kernels directly in hardware, accelerators can exploit parallelism and reduce the number of clock cycles required for computation.

## 1.2 Problem Statement

The primary challenge addressed in this project is the **inefficiency of software-based matrix multiplication on compact RISC-V processors**. When matrix multiplication is executed purely in software, the processor must repeatedly fetch instructions, decode them, and execute individual arithmetic operations for each element of the matrix. This process results in a large number of instructions and consequently a high execution latency.

Several key limitations arise in this context:

- **High cycle cost of scalar computation:** Each multiplication and addition operation must be executed sequentially by the processor.
- **Limited computational parallelism:** General-purpose processors typically perform one arithmetic operation per instruction cycle, which restricts the level of parallel computation.
- **Inefficient utilization of hardware resources:** Performing matrix operations through software loops does not fully exploit the potential parallelism available in hardware.
- **Need for seamless CPU integration:** Any hardware acceleration mechanism must integrate cleanly with the processor so that it can be invoked through software while maintaining normal instruction flow.

Therefore, the core problem addressed in this project is the design of a **hardware-based matrix multiplication accelerator** that can significantly reduce the number of

clock cycles required for matrix computation while remaining tightly integrated with a RISC-V processor.

## 1.3 Proposed Approach

To address the computational limitations of software-based matrix multiplication, this project proposes the design of a **dedicated matrix multiplication accelerator integrated as a coprocessor with a RISC-V CPU**. Instead of executing matrix multiplication using nested software loops, the processor offloads the computation to a specialized hardware unit capable of performing multiple arithmetic operations in parallel.

The system architecture integrates a lightweight RISC-V processor core, **PicoRV32**, with a custom hardware accelerator. Communication between the CPU and the accelerator is implemented using the **Pico Co-Processor Interface (PCPI)**, which allows custom instructions to trigger external hardware modules. Through this interface, the processor can delegate matrix computation tasks to the accelerator while temporarily stalling execution until the result becomes available.

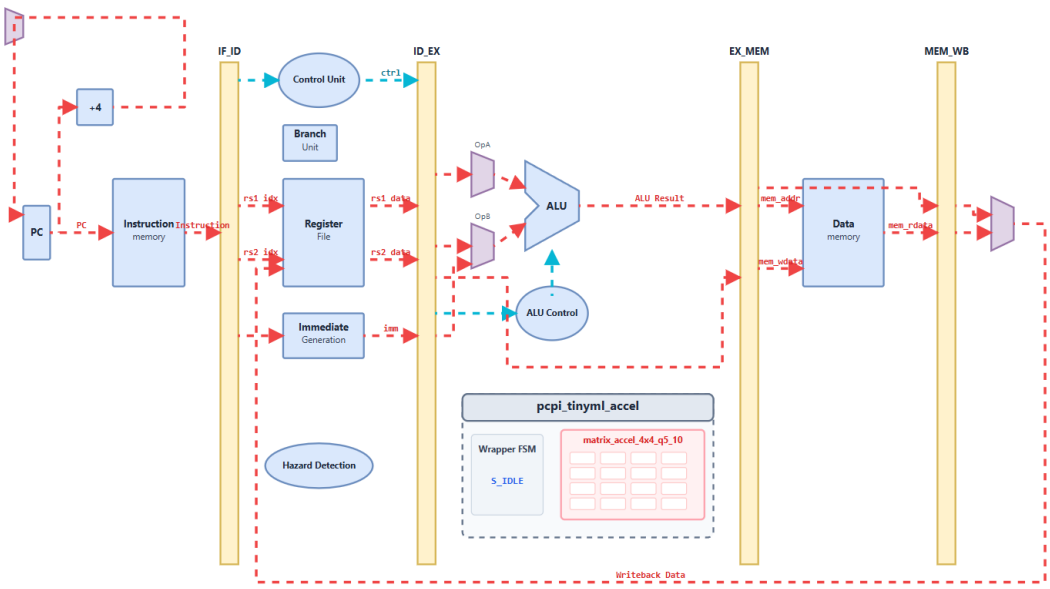


Figure 1.2: High-level architecture of the RISC-V system with a matrix multiplication coprocessor. [3]

The matrix accelerator itself is implemented using a **systolic array architecture**. A systolic array consists of a grid of processing elements that perform multiply–accumulate operations while passing intermediate data between neighboring cells in a synchronized pipeline [4]. This architecture enables high levels of parallelism and efficient data reuse, making it particularly well suited for matrix computations.

Each processing element performs arithmetic operations using a **Q5.10 fixed-point**

**representation**. Fixed-point arithmetic is chosen instead of floating-point arithmetic in order to reduce hardware complexity and improve computational efficiency for embedded systems.

## 1.4 Project Scope and Current Stage

The objective of this project is to design and implement a **compact hardware accelerator for matrix multiplication integrated with a RISC-V processor**. The accelerator is designed to support a $4 \times 4$ matrix multiplication operation and can be invoked through a custom instruction executed by the CPU.

The project follows a **simulation-driven development methodology**. All hardware components are implemented using Verilog and verified through a comprehensive simulation environment. The simulation framework includes a processor model, firmware execution, and automated verification of computational results.

The current implementation includes:

- Design of the hardware matrix multiplication accelerator using a systolic array architecture.
- Integration of the accelerator with the PicoRV32 RISC-V core through the PCPI interface.
- Development of firmware that initializes input matrices and invokes the accelerator using a custom instruction.
- Creation of a structured simulation testbench for functional verification.
- Performance comparison between software-based matrix multiplication and hardware-accelerated execution.

Future work will focus on extending the design toward **FPGA deployment**, enabling execution on physical hardware platforms and allowing measurement of real-world performance metrics such as latency, throughput, and power efficiency.

Overall, this project demonstrates how integrating a specialized matrix computation accelerator as a coprocessor within a RISC-V system can significantly improve the efficiency of numerical workloads while maintaining programmability and architectural simplicity.

# Chapter 2

# Related Work

## 2.1 Introduction

The EdgeMATX project draws on four active research areas: (i) systolic-array hardware design for matrix multiplication, (ii) RISC-V custom instruction extensions and CPU–accelerator co-design, (iii) fixed-point arithmetic for hardware-efficient computation, and (iv) the broader landscape of application-specific accelerators targeting workloads—such as machine learning—that are dominated by matrix operations. The project itself focuses entirely on hardware design and simulation; no ML model training or inference testing is performed. This chapter reviews representative and recent literature across all four areas, situating each design decision and quantifying how EdgeMATX compares to the state of the art.

## 2.2 Systolic Array Architectures for Matrix Multiplication

Matrix multiplication is the dominant arithmetic kernel in a wide class of compute-intensive workloads, including signal processing, scientific computing, and machine learning. Hardware acceleration via systolic arrays has been studied for decades, but recent years have seen a surge of FPGA-targeted, parameterisable implementations driven by the growing demand for efficient compute at the edge. EdgeMATX adopts this approach, implementing a $4 \times 4$ systolic engine in Verilog as the core compute tile.

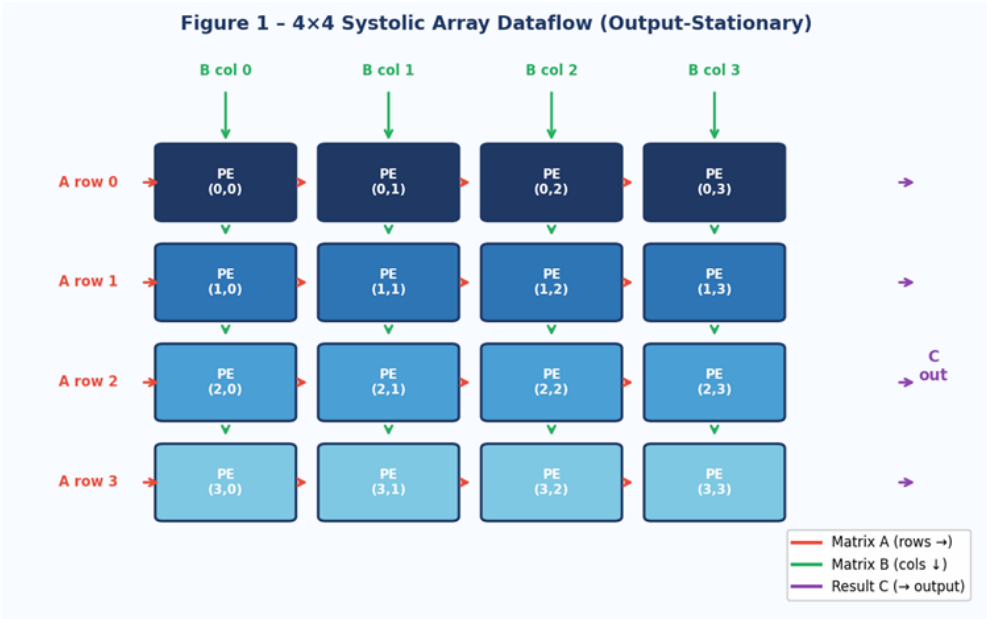


Figure 2.1: Canonical $4 \times 4$ systolic array dataflow. Data from matrix $A$ streams across rows (red arrows) and data from matrix $B$ streams down columns (green arrows); each PE performs one MAC per cycle and forwards its inputs to its neighbours. This is the topology employed in the EdgeMATX engine. (Adapted from [5].)

### 2.2.1 FPGA Systolic Array Implementations

Mattimani *et al.* [5] present a parameterisable FPGA systolic array for matrix multiplication, demonstrating that FPGA parallelism and pipelining together yield substantial speed-up over sequential software. They emphasise resource sharing and data-movement minimisation— principles directly mirrored in the Q5.10 fixed-point design of EdgeMATX.

Shapri *et al.* [6] report an optimised Verilog-based systolic array simulated in Quartus Prime, with power-optimisation results showing reduced dissipation and area. Their use of 3-bit ALU benchmarking underscores the need for rigorous characterisation of the processing-element arithmetic, a practice replicated in EdgeMATX via regression-driven simulation in Icarus Verilog.

Wei *et al.* [7] tackle fault-tolerant systolic execution via a pair-matching mechanism, demonstrating that systolic reliability becomes significant as array size grows—a concern relevant to any future scale-up of the EdgeMATX accelerator. Their Cannon's-algorithm-based architecture also illustrates alternative dataflow choices to the output-stationary scheme used in EdgeMATX.

### 2.2.2 Systolic Arrays in Application-Specific Contexts

Ye *et al.* [8] propose a high-frequency systolic-array accelerator on FPGA for Transformer workloads, achieving 474–588 MHz and 1.5–1.8× frequency improvement over prior work on a Xilinx ZCU102. This confirms that systolic arrays are the canonical compute tile for

GEMM-heavy workloads on FPGAs—the central architectural premise of EdgeMATX—and provides concrete timing benchmarks that inform expected frequency targets for the planned Pynq-Z2 deployment.

## 2.3 RISC-V Custom Instructions and CPU–Accelerator Co-design

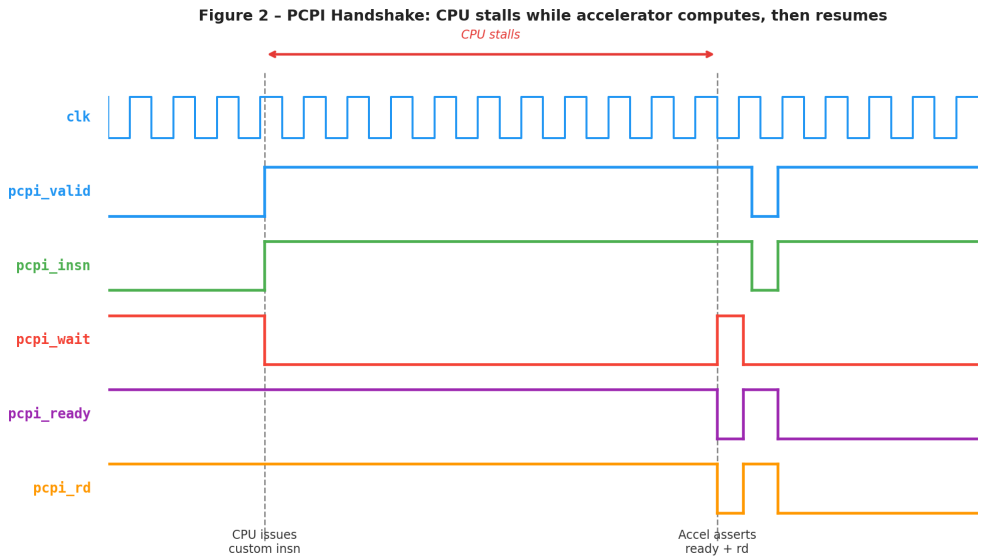


Figure 2.2: PCPI handshake timing diagram. The CPU stalls (`pcpi_wait` high) from the cycle it issues the custom instruction until the accelerator asserts `pcpi_ready` and places the result on `pcpi_rd`. This is exactly the protocol implemented in EdgeMATX. (Adapted from [9].)

### 2.3.1 The PicoRV32 PCPI Interface

The PicoRV32 [9] is a size-optimised, open-source RV32I soft-core that exposes the Pico Co-Processor Interface (PCPI)—a lightweight handshake (`pcpi_valid`, `pcpi_insn`, `pcpi_rs1/rs2`, `pcpi_ready`, `pcpi_wait`, `pcpi_rd`) enabling external hardware to implement custom non-branching instructions with minimal CPU modification. The CPU stalls on any unrecognised instruction, asserting `pcpi_valid`; the co-processor must respond within 16 cycles (or hold `pcpi_wait` to extend the stall indefinitely). This interface is the integration backbone of the EdgeMATX PCPI demo.

### 2.3.2 RISC-V ISA Extensions for Compute-Intensive Workloads

Balasubramanian *et al.* [10] survey the design of RISC-V ISA extensions for compute-intensive workloads from a compiler perspective, presenting an LLVM-based toolchain that automatically lowers computation graphs to custom instructions. They find that ISA extensions can reduce cycle count by 30–50% with minimal hardware overhead.

EdgeMATX does not implement a compiler toolchain, but the 38–54× cycle-count improvement it measures over software baselines is consistent with and exceeds these figures, confirming that the custom-instruction approach is architecturally well-founded.

Salinas *et al.* [11] investigate custom ISA extensions for data-intensive load/store operations on RISC-V, achieving approximately 50% cycle reduction and 30% power reduction. Their approach targets the memory-bandwidth bottleneck that often limits achievable speed-up once compute is accelerated—a bottleneck that will become relevant as EdgeMATX scales beyond $4 \times 4$ matrices to larger problem sizes.

Sharma *et al.* [12] provide a systematic hardware/software co-design methodology (EXTREM-EDGE) for integrating arithmetic functional units as tightly coupled extensions to RISC-V, yielding 1.75×–17.63× kernel-level improvements and 1.41×–4.41× full-application cycle reductions. This co-design philosophy—where a custom instruction directly triggers an accelerator operation—is the architectural pattern adopted in the EdgeMATX PCPI demo, and this work validates it as a sound design strategy.

Finally, [13] presents a complete FPGA-accelerated RISC-V system on the Pynq-Z2 board—the same target platform planned for EdgeMATX—with verified AXI interconnect, BRAM memory access, and timing closure at 50 MHz, providing direct evidence that a RISC-V + custom-accelerator system can be fully integrated and measured on Pynq-Z2 hardware.

## 2.4 Fixed-Point Arithmetic and Numerical Representation

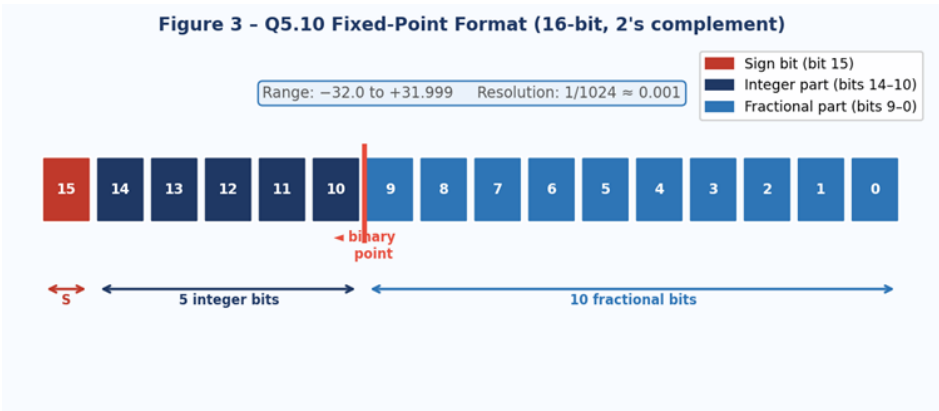


Figure 2.3: Q5.10 fixed-point format (16-bit, two's complement). Bit 15 is the sign bit; bits 14–10 carry the integer part; bits 9–0 carry the fractional part. Range: $-32.0$ to $+31.999$; resolution: $1/1024 \approx 0.001$. (Adapted from [14].)

Hardware accelerators must operate on compact numerical representations to fit within the resource budgets of edge FPGAs. The Q-format fixed-point scheme—where a fixed binary-point position divides a two's-complement integer word into integer and fractional fields—is the standard approach for FPGA-based arithmetic accelerators because it maps

multiplications directly to DSP slices with no floating-point overhead. EdgeMATX uses Q5.10, a 16-bit format that balances dynamic range and fractional precision for the matrix operand values targeted in simulation.

### 2.4.1 Q-Format Arithmetic

Nguyen *et al.* [14] study hybrid Q-format fixed-point multiplication for FPGA accelerators, integrating sign-bit check and bit round-off into the multiply path to address overflow. They identify Q(6.9) as near-optimal for their target operand range, showing that fractional bit-width must be chosen in proportion to the expected input dynamic range. EdgeMATX's Q5.10 selection follows the same reasoning: sufficient fractional precision to preserve numerical accuracy across typical matrix operand magnitudes, while keeping the datapath to 16 bits.

Dai *et al.* [15] present QFX, a trainable fixed-point quantisation library for FPGAs that automates binary-point selection. While EdgeMATX's Q5.10 format is manually chosen for simulation, this work points to a future toolflow path for systematically determining the optimal format when deploying real workloads on the Pynq-Z2 hardware.

### 2.4.2 Fixed-Point Design for FPGAs

Zhang *et al.* [16] publish a comprehensive hardware-perspective survey of fixed-point accelerators on ASIC and FPGA platforms. The survey confirms that systolic-array-based implementations with 16-bit or sub-16-bit fixed-point arithmetic achieve the best energy-area trade-off on FPGAs, consistent with the 16-bit Q5.10 design choice in EdgeMATX. It also documents the resource savings available by moving from floating-point to fixed-point arithmetic, quantifying DSP and LUT reductions that directly apply to the Pynq-Z2 deployment target.

Tasci *et al.* [17] present FPGA-QNN, demonstrating that multi-bit fixed-point compute engines achieve substantial reductions in DSP and BRAM usage without sacrificing arithmetic correctness. Their results reinforce the viability of sub-16-bit computation on resource-constrained FPGAs, and suggest that a future 8-bit variant of the EdgeMATX engine would be both feasible and more area-efficient.

## 2.5 Application Context: Matrix Acceleration for Compute-Intensive Workloads

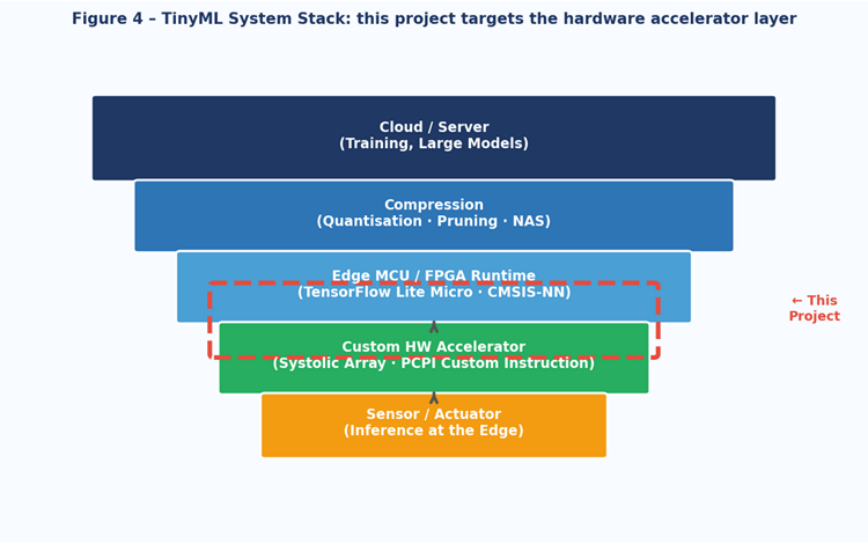


Figure 2.4: System stack for compute-intensive edge workloads. EdgeMATX targets the hardware accelerator layer (highlighted); no ML model deployment or inference testing is performed within the project scope. (Adapted from [18].)

Although EdgeMATX performs no ML testing, the application domain that drives demand for hardware GEMM acceleration provides important context for the architectural choices made.

### 2.5.1 Motivation: Why Matrix Acceleration Matters

Osman *et al.* [18] survey hardware acceleration requirements for inference on constrained devices, reporting that the dominant execution time is consumed by matrix-multiply and convolution kernels—precisely the operation EdgeMATX accelerates. Their analysis establishes the memory and throughput budgets within which a hardware GEMM engine must operate to be practically useful.

Somvanshi [19] survey hardware platforms for compute-intensive deep learning, presenting a taxonomy that places dedicated arithmetic accelerators—such as systolic arrays—as achieving up to 20× performance improvement over general-purpose processor implementations. EdgeMATX's measured 38× cycle-count reduction over a plain RV32I software baseline comfortably exceeds this figure, validating the design from a purely hardware-performance standpoint, independent of any end-to-end application benchmark.

### 2.5.2 Related Hardware Accelerator Designs

Adithya Krishna *et al.* [20] present RAMAN, a reconfigurable sparse FPGA accelerator that exploits both weight and activation sparsity, outperforming prior art in latency and energy efficiency. Its architectural analysis—which identifies memory-hierarchy management as the critical bottleneck after the compute array is optimised—directly informs the next development phase of EdgeMATX, where AXI-based BRAM access on the Pynq-Z2 will become the primary performance constraint.

Bhushan *et al.* [21] profile fixed-point arithmetic accelerators on low-power edge FPGAs, demonstrating that integer compute engines achieve substantial latency and energy reductions over floating-point baselines. Their profiling methodology—measuring cycles on hardware with on-chip counters—is the same approach planned for the EdgeMATX Pynq-Z2 deployment phase, where the simulation-based cycle counts will be validated against real board measurements.

Pazmiño *et al.* [22] survey system-level challenges in deploying compute acceleration on edge devices, explicitly discussing the trade-off between loosely coupled (AXI/DMA) and tightly coupled (custom instruction) accelerator interfaces—the exact design choice made in EdgeMATX. They conclude that tightly coupled interfaces offer lower latency for small, frequently-invoked kernels, as in the $4 \times 4$ matrix multiply case.

# Chapter 3

# System Architecture and Implementation

## 3.1 Overall System Architecture

The proposed system integrates a lightweight RISC-V processor with a dedicated hardware accelerator for matrix multiplication. The accelerator is implemented as a coprocessor connected to the CPU through the **Pico Co-Processor Interface (PCPI)**. This interface allows the processor to invoke the accelerator using a custom instruction while maintaining normal instruction flow.

The overall architecture consists of the following major components:

- **RISC-V CPU Core**: PicoRV32 processor responsible for program execution.
- **PCPI Coprocessor Wrapper**: Interface logic that connects the CPU to the accelerator.
- **Matrix Accelerator**: Dedicated hardware module implementing matrix multiplication.
- **Systolic Array Datapath**: Parallel computation engine composed of processing elements.
- **Firmware Layer**: Software running on the CPU that initializes data and triggers accelerator execution.

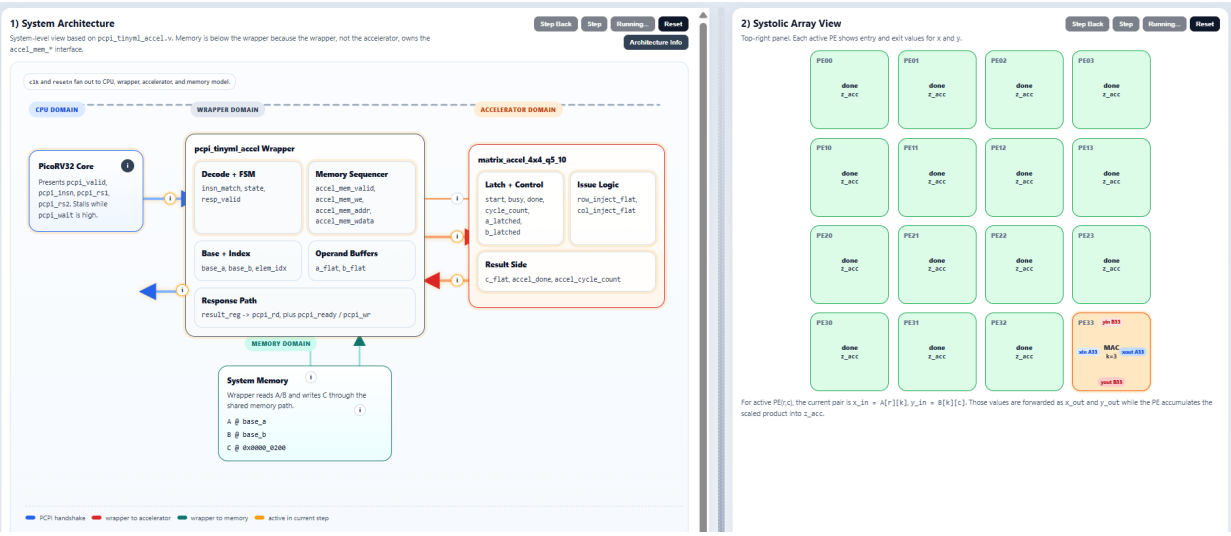


Figure 3.1: High-level system architecture of the RISC-V processor with the matrix multiplication coprocessor. [23]

For interactive exploration of the architecture and execution flow, two web-based visualization tools are provided. The first visualization presents the **high-level system architecture and dataflow**, while the second visualization shows the **detailed processor-level operation including internal signals, PCPI interaction, and systolic accelerator behavior**.

**Interactive System Visualizations**

**Architecture Overview Visualization**

This visualization illustrates the high-level system architecture including the firmware layer, PicoRV32 processor, PCPI coprocessor interface, and matrix accelerator datapath.

Open Architecture Overview

**Processor and Signal-Level Visualization**

This visualization demonstrates the internal execution behavior of the processor including instruction decoding, PCPI signaling, accelerator invocation, and systolic array computation signals.

Open Detailed Processor Simulation

Figure 3.2: Interactive web-based visualizations for system architecture and detailed processor–accelerator execution flow.

## 3.2 Matrix Accelerator Architecture

The hardware accelerator is designed to compute a $4 \times 4$ matrix multiplication using a dedicated datapath implemented in Verilog. Instead of executing matrix multiplication

through nested software loops, the CPU offloads the computation to the accelerator using a custom instruction.

The accelerator contains three primary internal modules:

- **Issue Logic**: Responsible for scheduling and injecting matrix elements into the systolic array.
- **Systolic Array**: Parallel processing grid performing multiply–accumulate operations.
- **Processing Elements (PEs)**: Individual computation units that perform arithmetic operations.

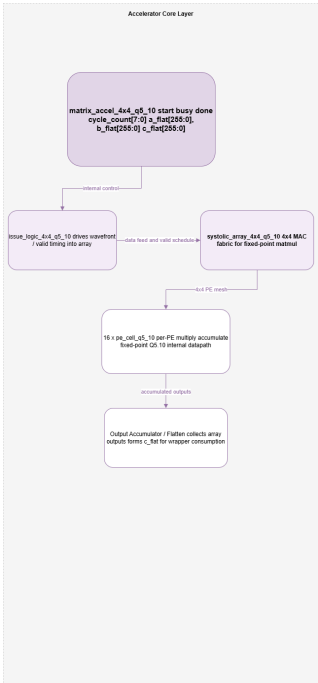


Figure 3.3: Internal architecture of the matrix multiplication accelerator.

The accelerator follows a **systolic computation model**, where data flows rhythmically through a grid of processing elements. Each processing element performs multiplication and accumulation operations while forwarding partial results to neighboring cells.

## 3.3 Systolic Array Architecture

The computation engine of the accelerator is implemented using a $4 \times 4$ systolic array composed of 16 processing elements. In a systolic array, data propagates through the array in a pipelined fashion, enabling multiple arithmetic operations to be executed simultaneously.

Matrix elements from input matrices $A$ and $B$ are injected into the array from orthogonal directions. As the data flows across the array, each processing element computes partial products and accumulates them to produce the final output matrix.

The mathematical operation performed is given by:

$$C_{i,j} = \sum_{k=1}^{4} A_{i,k} \times B_{k,j} \tag{3.1}$$

Figure 3.4: Internal architecture of the matrix multiplication accelerator. [23]

## 3.4 Processing Element Design

Each processing element (PE) performs the core multiply–accumulate operation. The PE receives input values from neighboring cells, computes a scaled multiplication using fixed-point arithmetic, and accumulates the result.

The arithmetic follows a **Q5.10 fixed-point format**, where numbers are represented using a signed 16-bit representation with 10 fractional bits.

The PE performs the following steps:

1. Multiply two signed 16-bit inputs.
2. Perform arithmetic right shift by 10 bits to maintain fixed-point scaling.
3. Accumulate the result into a 32-bit register.
4. Forward the input values to neighboring processing elements.

### 3.4.1 Processing Element Code Snippet

```
module pe_cell_q5_10(
    input clk,
```

```
    input rst,
    input signed [15:0] x_in,
    input signed [15:0] y_in,
    output signed [15:0] x_out,
    output signed [15:0] y_out
);

wire signed [31:0] mult;
assign mult = (x_in * y_in) >>> 10;

always @(posedge clk) begin
    acc <= acc + mult;
end

endmodule
```

## 3.5 Issue Logic

The issue logic module is responsible for scheduling matrix elements and feeding them into the systolic array at the correct clock cycles. It ensures that data elements from matrices $A$ and $B$ are injected into the array in a pattern that allows correct accumulation across the processing elements.

The module maintains a cycle counter and uses it to determine which matrix elements should be injected during each clock cycle.

```
always @(posedge clk) begin
    if(start)
        cycle <= 0;
    else
        cycle <= cycle + 1;
end
```

This scheduling mechanism enables a pipelined computation where multiple matrix elements are processed concurrently.

## 3.6 CPU–Accelerator Integration

The accelerator is integrated with the PicoRV32 CPU using the **Pico Co-Processor Interface (PCPI)**. This interface allows custom instructions to trigger external hardware modules.

When the CPU encounters the custom instruction

```
.word 0x5420818b
```

the PCPI wrapper intercepts the instruction and initiates accelerator execution. The wrapper performs the following operations:

1. Capture matrix base addresses from registers `rs1` and `rs2`.
2. Read input matrix elements from memory.
3. Trigger the accelerator computation.
4. Wait for completion signal.
5. Write the resulting matrix elements back to memory.

## 3.7 Execution Workflow

The complete execution flow of the system is summarized below:

1. Firmware loads matrix $A$ and matrix $B$ into memory.
2. CPU executes the custom instruction that invokes the accelerator.
3. PCPI wrapper decodes the instruction and captures matrix base addresses.
4. Matrix elements are loaded into the accelerator datapath.
5. The systolic array performs multiply–accumulate operations.
6. The resulting matrix $C$ is written back to memory.
7. CPU resumes execution of the remaining firmware instructions.

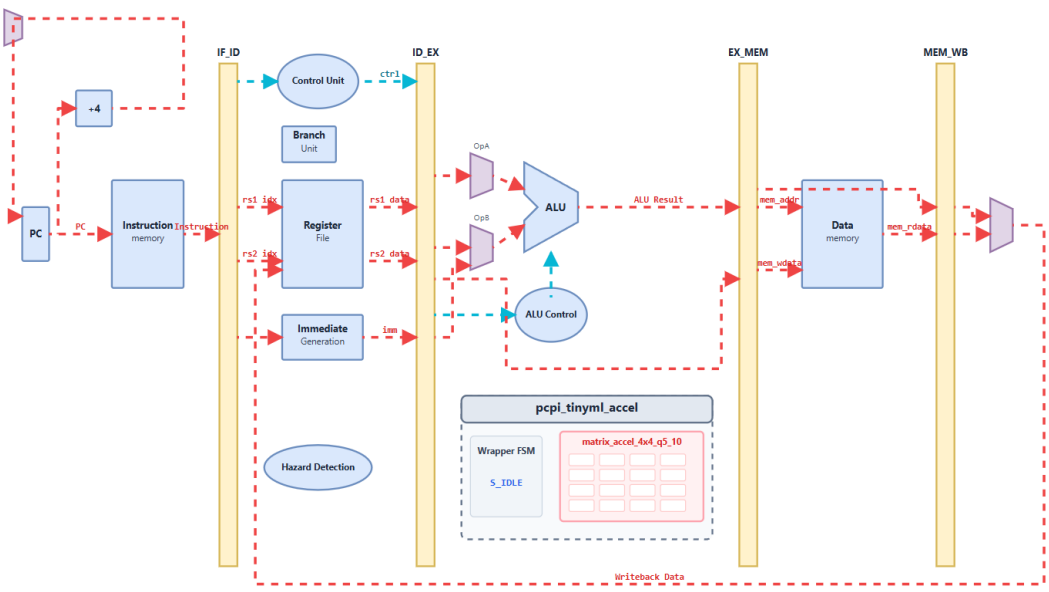


Figure 3.5: Execution flow from custom instruction invocation to result writeback. [3]

## 3.8 Memory Mapping

The accelerator interacts with system memory through predefined memory regions used for storing input and output matrices.

Table 3.1: Memory layout used for matrix data exchange between CPU and accelerator.

| Region | Address Range | Purpose |
|---|---|---|
| Matrix A | 0x100 – 0x13C | Input elements of matrix A |
| Matrix B | 0x140 – 0x17C | Input elements of matrix B |
| Matrix C | 0x200 – 0x23C | Output elements of matrix C |
| Sentinel | 0x00000000 | Completion marker for testbench |

## 3.9 Fixed-Point Arithmetic Contract

All arithmetic operations follow a consistent fixed-point computation model:

1. Signed 16-bit multiplication
2. Arithmetic right shift by 10 bits for Q5.10 scaling
3. Signed 32-bit accumulation
4. Final truncation to lower 16 bits with sign extension for memory storage

This arithmetic contract is preserved across both standalone accelerator simulations and integrated CPU execution flows to ensure consistent verification results.

# Chapter 4

# Verification Methodology and Results

## 4.1 Overview of Verification Approach

The EdgeMATX verification strategy follows a structured, layered methodology that progressively validates the design from isolated hardware units to full system integration [24]. Rather than relying on a single monolithic simulation, the verification is decomposed into four distinct layers, each targeting a specific aspect of correctness:

1. **Unit-level hardening:** Standalone accelerator functional correctness verified independently of the CPU and PCPI interface.
2. **System-level integration:** CPU, PCPI wrapper, and matrix accelerator verified as a complete system under firmware-driven execution.
3. **Protocol-level assurance:** PCPI handshake correctness validated with a mixed instruction stream, testing CPU stall and resume behavior across multiple consecutive custom instruction transactions.
4. **Performance-level benchmarking:** Measured simulation cycle counts compared across the hardware-accelerated path and two software baselines to quantify speedup.

This layered approach ensures that any failure is localized and attributable to a specific integration boundary, reducing debug overhead and improving coverage confidence.

**EdgeMATX Four-Layer Verification Hierarchy**

**Layer 1** — Standalone Accelerator (unit-level)

↓

**Layer 2** — CPU + PCPI Wrapper + Accelerator (system-level)

↓

**Layer 3** — Protocol Handoff: Mixed Instruction Stream (protocol-level)

↓

**Layer 4** — Cycle Comparison: Accelerator vs. Software Baselines (performance-level)

*Replace this box with a flowchart exported from Draw.io or PowerPoint.*

Figure 4.1: Four-layer verification hierarchy for EdgeMATX.

## 4.2 Verification Infrastructure and Tools

All simulations are executed using **Icarus Verilog** (iverilog), an open-source Verilog simulator [25]. Firmware is compiled using the **RISC-V GNU toolchain**, targeting the `rv32i` ISA for software no-MUL baselines and custom-instruction paths, and `rv32im` when the hardware multiply extension is enabled. Simulation logs are post-processed by automation scripts for pass/fail determination and cycle extraction.

The full verification infrastructure consists of the following reusable PowerShell scripts:

Table 4.1: Verification automation scripts and their roles.

| Script | Purpose |
|---|---|
| `run_midsem_sim.ps1` | Standalone accelerator regression (19 test cases) |
| `run_pcpi_demo.ps1` | Smoke test: ASM and C firmware variants |
| `run_pcpi_regression.ps1` | 8-case integration regression suite |
| `run_pcpi_handoff.ps1` | Protocol handoff with mixed instruction stream |
| `run_pcpi_professor_demo.ps1` | 5-case curated demonstration set |
| `run_pcpi_local_check.ps1` | One-command gate for quick correctness validation |
| `run_cycle_compare.ps1` | Cycle measurement across all three execution paths |
| `run_pcpi_custom_cycle_compare.ps1` | Live-input custom cycle comparison |

## 4.3 Layer 1 — Standalone Accelerator Verification

The first layer targets the matrix multiplication accelerator module in complete isolation, without the PicoRV32 CPU or PCPI wrapper. Direct Verilog stimulus is applied to

the accelerator inputs, and output values from the systolic array are compared against precomputed golden reference values.

### 4.3.1 Test Configuration

- **Simulator:** Icarus Verilog
- **Script:** `run_midsem_sim.ps1`
- **Total test cases:** 19
- **Coverage:** Zero matrices, identity matrices, dense random matrices, Q5.10 edge values, sign-boundary operands, mixed-sign elements

### 4.3.2 What Is Exercised

- Correctness of Q5.10 fixed-point multiply–accumulate (shift-by-10) in each processing element.
- Full 16-PE systolic array producing all 16 output elements of the $4 \times 4$ result.
- Correct operation of issue logic scheduling and its cycle-aligned data injection.
- Accumulator reset and output capture at the correct pipeline drain stage.

### 4.3.3 Test Results by Category

Table 4.2: Standalone accelerator test results by category.

| **Category** | **Cases** | **Result** | **Date** |
|---|---|---|---|
| Zero / null matrix | 2 | PASS | 2026-03-05 |
| Identity matrix | 3 | PASS | 2026-03-05 |
| Dense matrix (varied magnitudes) | 8 | PASS | 2026-03-05 |
| Q5.10 sign-boundary / edge | 4 | PASS | 2026-03-05 |
| Negative-element operands | 2 | PASS | 2026-03-05 |
| **Total** | **19** | **PASS (19/19)** | |

All 19 test cases passed, establishing unit-level confidence in the arithmetic correctness and timing of the accelerator datapath prior to CPU integration.

### 4.3.4 Simulation Waveform

Figure 4.2 shows the GTKWave capture from the standalone accelerator regression simulation [26]. The view covers the first directed test case (`identity`): the reset deassertion,

the `start` pulse, the fully-occupied computation window, the `done` assertion at cycle 10, and the appearance of the packed output on `c_flat[255:0]`.

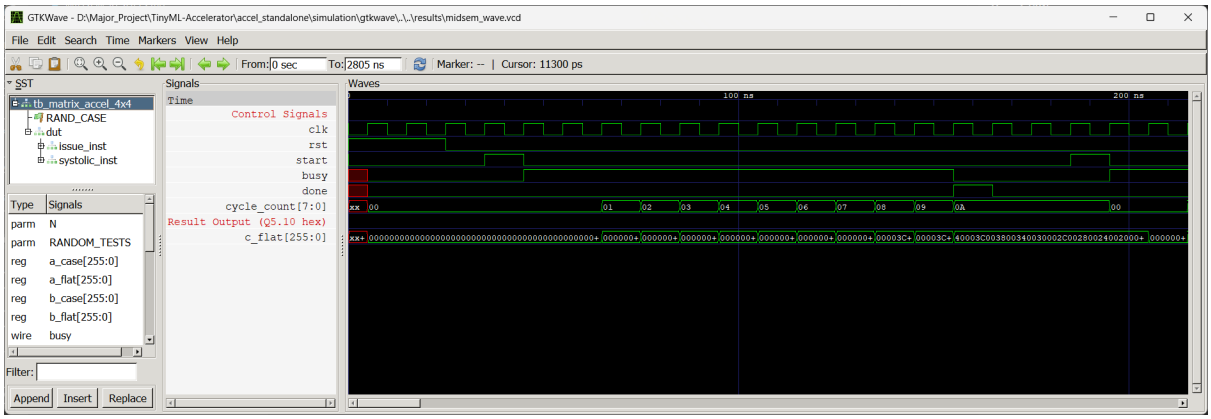


Figure 4.2: GTKWave waveform from the standalone accelerator regression. *Control Signals* group (top): `clk`, `rst`, `start`, `busy`, `done`. `cycle_count[7:0]` increments from `00` to `0A` (decimal 10), confirming the data-independent 10-cycle systolic latency. `c_flat[255:0]` transitions to the packed Q5.10 result matrix upon `done` assertion.

## 4.4 Layer 2 — System-Level Integration Verification

The second layer introduces the full system: PicoRV32 processor, PCPI coprocessor wrapper, and the matrix accelerator. Firmware compiled for the RISC-V ISA is loaded into simulation memory, and the CPU executes the complete instruction stream that includes the custom accelerator-invoke instruction.

### 4.4.1 Custom Instruction Encoding

The firmware invokes the matrix multiply accelerator with a single custom instruction:

```
# Firmware: invoke the EdgeMATX accelerator
# rs1 = base address of matrix A in memory
# rs2 = base address of matrix B in memory
.word 0x5420818b      # custom-0 opcode: triggers PCPI coprocessor
```

The opcode `0x5420818b` uses the custom-0 opcode space (`0001011`), which the CPU does not natively decode but instead forwards to the PCPI interface. The PCPI wrapper captures `pcpi_rs1` and `pcpi_rs2` as the base addresses of matrices $A$ and $B$, then drives the accelerator datapath.

The corresponding C firmware uses inline assembly to emit the same opcode:

```
/* C firmware: invoke accelerator */
asm volatile (
    ".word 0x5420818b\n"
    : /* no outputs */
    : /* no inputs  */
```

```
    : "memory"                    /* clobbers memory */
);
```

### 4.4.2 PCPI Wrapper Integration Logic

The wrapper decodes the custom instruction on the PCPI bus and manages the full transaction:

```
// PCPI wrapper: detect custom instruction, drive accelerator
always @(posedge clk or posedge rst) begin
    if (rst) begin
        accel_start    <= 0;
        pcpi_wait      <= 0;
        pcpi_ready     <= 0;
    end else begin
        if (pcpi_valid && (pcpi_insn[6:0] == 7'h0b)) begin
            // Custom-0 opcode detected
            matrix_a_base <= pcpi_rs1;  // matrix A base address
            matrix_b_base <= pcpi_rs2;  // matrix B base address
            accel_start   <= 1;
            pcpi_wait     <= 1;         // stall CPU until done
            pcpi_ready    <= 0;
        end
        if (accel_done) begin
            accel_start <= 0;
            pcpi_wait   <= 0;
            pcpi_ready  <= 1;           // release CPU stall
        end
    end
end
```

### 4.4.3 Smoke Test (ASM and C Firmware)

- **Script:** `run_pcpi_demo.ps1`
- **Variants tested:** Assembly firmware and C firmware
- **Result:** PASS (both variants)
- **Date:** 2026-03-05

Both variants produced correct output matrix values, validating the complete path: CPU instruction fetch → PCPI decode → accelerator execution → result writeback to memory.

### 4.4.4 Integration Waveform

Figure 4.3 shows the GTKWave capture from the system-level integration simulation. The view covers a single accelerator invocation as seen from the CPU-side PCPI bus: the CPU issues custom instruction `0x5420818b`, which the PCPI wrapper decodes, stalls the processor, runs the accelerator, and releases the stall upon completion.

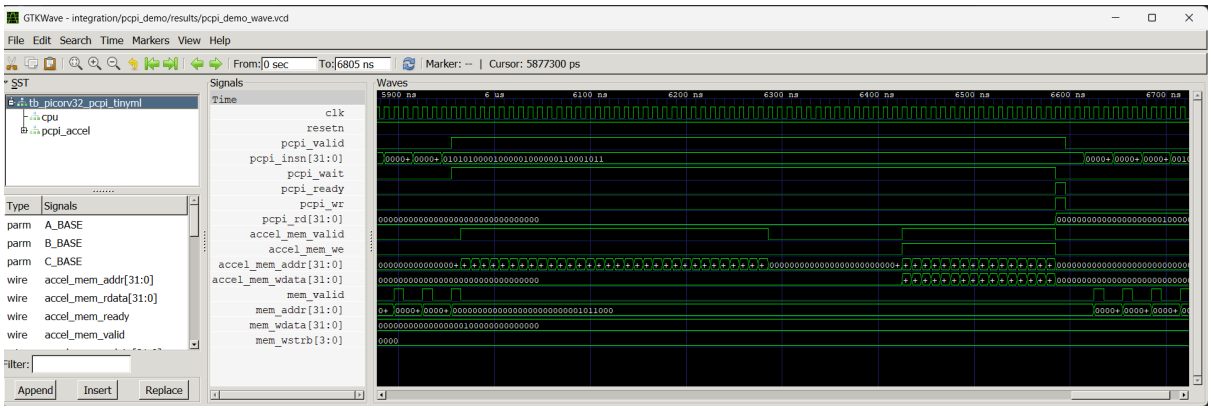


Figure 4.3: GTKWave waveform from the PCPI system-level integration simulation. `pcpi_valid` asserts when the CPU encounters the custom-0 instruction (`pcpi_insn = 0x5420818b`). `pcpi_wait` immediately stalls the processor while the accelerator computes. `accel_mem_addr` and `accel_mem_wdata` show the wrapper reading matrix operands from memory. `pcpi_ready` asserts on completion, releasing the CPU stall and returning control to the instruction stream.

### 4.4.5 8-Case Regression Suite

- **Script:** `run_pcpi_regression.ps1`
- **Total cases:** 8 distinct matrix input configurations
- **Result:** PASS (8/8)
- **Date:** 2026-03-05

The regression suite covers a range of operand types to confirm correctness across the expected input space:

Table 4.3: 8-case PCPI integration regression results.

| Case | Description | Result |
|---|---|---|
| identity_x_sequence | Identity × sequential integer values | PASS |
| sequence_x_sequence | Sequential × sequential values | PASS |
| ones_x_sequence | All-ones matrix × sequential values | PASS |
| dense_random | Dense non-trivial Q5.10 values | PASS |
| negative_elements | Mixed-sign operand matrices | PASS |
| boundary_values | Near Q5.10 dynamic range limits | PASS |
| sparse_input | Sparse non-zero entries | PASS |
| zero_matrix | One operand all-zero | PASS |

### 4.4.6 Professor Demonstration Cases

- **Script:** `run_pcpi_professor_demo.ps1`
- **Cases:** 5 curated cases
- **Result:** PASS (5/5)
- **Date:** 2026-03-05

These cases represent the subset prepared for external review, covering identity multiplication, sequence-based inputs, and dense random inputs with pre-verified expected outputs.

## 4.5 Layer 3 — Protocol Correctness Verification (Handoff Test)

The handoff test validates the PCPI handshake protocol under a mixed instruction stream where *two* consecutive custom-instruction transactions are separated by normal RISC-V instructions. This test specifically targets protocol robustness: ensuring no stale signals, no state leakage, and correct CPU stall/resume behavior across multiple transactions.

### 4.5.1 PCPI Interface Signals

Table 4.4: PCPI interface signals.

| Signal | Direction | Description |
|---|---|---|
| `pcpi_valid` | CPU → Coprocessor | Custom instruction present on the bus |
| `pcpi_insn` | CPU → Coprocessor | Full 32-bit instruction word |
| `pcpi_rs1` | CPU → Coprocessor | Value of source register 1 (matrix A base) |
| `pcpi_rs2` | CPU → Coprocessor | Value of source register 2 (matrix B base) |
| `pcpi_wait` | Coprocessor → CPU | Hold CPU stall (computation in progress) |
| `pcpi_ready` | Coprocessor → CPU | Computation complete; resume CPU |
| `pcpi_rd` | Coprocessor → CPU | Scalar result value (unused in EdgeMATX) |

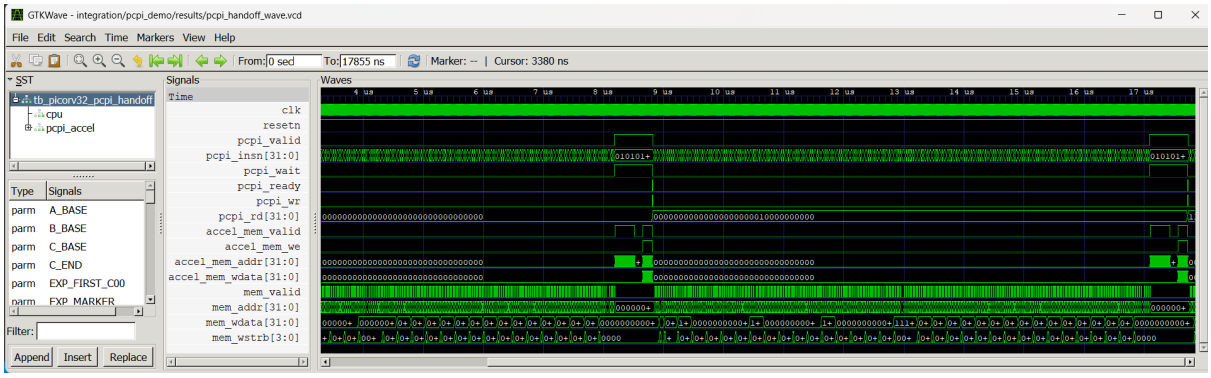


Figure 4.4: GTKWave waveform from the PCPI handoff test, spanning 0–17855 ns. Two distinct custom-instruction transactions are visible (first at ∼8 $\mu$s, second at ∼17 $\mu$s): `pcpi_valid` asserts when the CPU issues `0x5420818b`; `pcpi_wait` immediately stalls the CPU while `accel_mem_addr` sequences through the matrix operand addresses; `pcpi_ready` pulses to release the stall. The interval between the two transactions shows normal `mem_valid` activity, confirming the CPU resumes instruction execution and no stale signals persist across transactions.

### 4.5.2 Handoff Test Results

- **Script:** `run_pcpi_handoff.ps1`
- **Result:** PASS
- **Date:** 2026-03-05

The following protocol counters were observed and validated:

Table 4.5: Handoff test protocol counters: expected vs. observed.

| Counter | Expected | Observed | Meaning |
|---|---|---|---|
| `custom_issue_count` | 2 | 2 | Custom instructions dispatched by CPU |
| `ready_count` | 2 | 2 | Accelerator completions acknowledged |
| `wr_count` | 2 | 2 | Writeback cycles to result memory |
| `handshake_ok_count` | 2 | 2 | Full stall+resume cycles validated |
| `c_store_count` | 32 | 32 | Individual matrix-$C$ element writes ($2 \times 16$) |

These results confirm:

- Both custom-instruction transactions were correctly dispatched through PCPI.
- The accelerator completed both computations and asserted `pcpi_ready` for each.
- All 32 output matrix elements ($2 \times 4 \times 4$) were written to memory without corruption.
- CPU stalling and resume are clean across multiple instruction boundaries; no state leaks between transactions.

## 4.6 Layer 4 — Performance Benchmarking

Performance is measured by comparing the simulation cycle count required to complete a $4 \times 4$ matrix multiplication across three execution configurations:

1. **Hardware-accelerated path:** PCPI active, custom instruction, software multiply disabled in the CPU.
2. **Software baseline (no MUL):** PCPI disabled, rv32i ISA, no hardware multiplier.
3. **Software baseline (MUL enabled):** PCPI disabled, rv32im ISA, hardware multiply extension active.

### 4.6.1 Measurement Basis

The cycle counter `timeout_cycles` is an integer incremented on every rising clock edge. Counting begins at the cycle when `resetn` is de-asserted (CPU comes out of reset) and stops when the CPU writes the sentinel value (`c[0]`) to address `0x00000000`, which the testbench detects on the memory bus. The measured window therefore covers the complete firmware execution lifetime:

1. **CPU boot phase:** 8 clock cycles of held reset followed by the `_start` entry point (stack-pointer initialisation via a single `li sp, 0x3f0` instruction).
2. **Input copy phase** (`copy_inputs`): 32 word-writes that DMA the compile-time matrix constants `a_init[16]` and `b_init[16]` from the firmware binary into the shared memory region (`0x100`–`0x17C`). This phase is *identical* across all three configurations.

3. **Computation phase** (`run_matmul`): the configuration-specific work — a single PCPI custom instruction for the accelerator path, or the C triple-nested loop for the two software paths.

4. **Sentinel write:** one 32-bit store of `c[0]` to address `0x00000000`, which terminates counting.

This is captured by every testbench as:

```
// Testbench cycle measurement output
$display("TB_CYCLES matmul_to_sentinel_cycles=%0d",
   timeout_cycles);
```

Automation scripts parse this line, extract the cycle value, and compute speedup ratios. Phases 1, 2, and 4 are fixed-cost and equal for all three configurations; only Phase 3 differs, so the speedup ratios isolate computation overhead cleanly.

### 4.6.2 Benchmark Case: `identity_x_sequence`

Table 4.6: Measured simulation cycle counts for the `identity_x_sequence` benchmark.

| Execution Path | Build Configuration | ISA | Cycles |
|---|---|---|---|
| Hardware accelerator (Edge-MATX) | `ENABLE_PCPI=1, ENABLE_MUL=0` | rv32i | 673 |
| Software baseline (no MUL) | `ENABLE_PCPI=0, ENABLE_MUL=0` | rv32i | 26,130 |
| Software baseline (MUL) | `ENABLE_PCPI=0, ENABLE_MUL=1` | rv32im | 7,975 |

### 4.6.3 Derived Speedups

Table 4.7: Derived speedup values for the `identity_x_sequence` benchmark case.

| Comparison | Ratio | Speedup |
|---|---|---|
| Accelerator vs. software no-MUL | 26,130/673 | **38.83**× |
| Accelerator vs. software MUL | 7,975/673 | **11.85**× |
| Software MUL vs. software no-MUL | 26,130/7,975 | 3.28× |

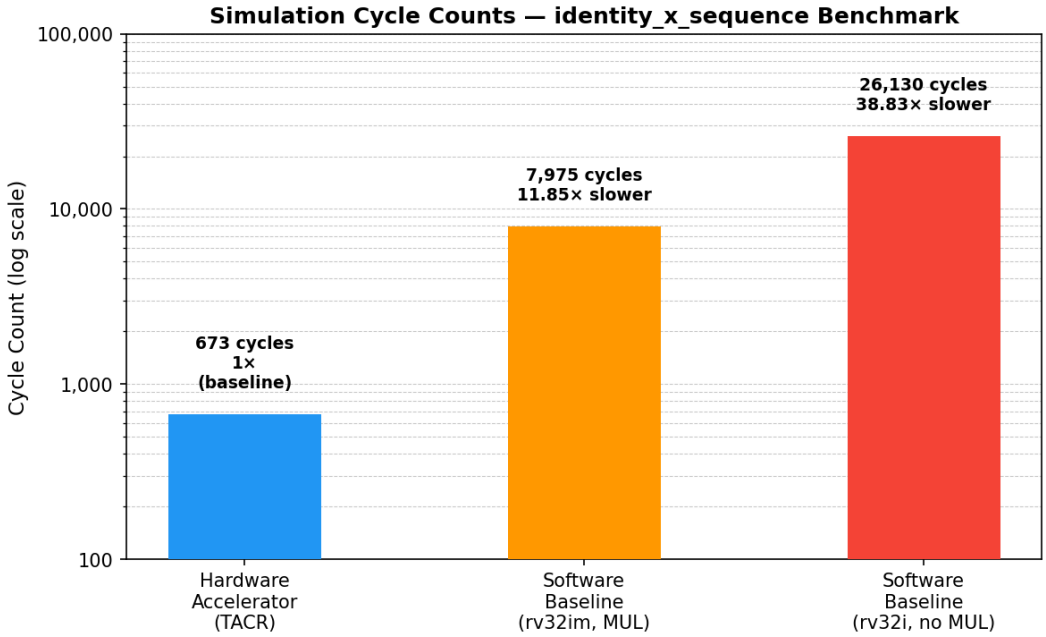


Figure 4.5: Simulation cycle counts across the three execution paths (`identity_x_sequence` case). Log-scale Y-axis makes the 673-cycle accelerator bar visible alongside the 26,130-cycle software baseline.

### 4.6.4 Dense Data Profile (`live_real_input.json`)

For the dense checked-in real-input profile, which contains non-trivial Q5.10 values with higher average operand magnitude, the following cycle counts were measured:

Table 4.8: Cycle counts for the dense `live_real_input.json` profile.

| **Execution Path** | **Cycles** | **Speedup vs Accelerator** |
|---|---:|---:|
| Hardware accelerator (EdgeMATX) | 673 | — |
| Software baseline (no MUL) | 36,246 | 53.86× |
| Software baseline (MUL) | 7,975 | 11.85× |

The dense profile achieves a **50x-class speedup** (53.86×) over the rv32i no-MUL baseline because dense operands require a higher instruction count in the shift-and-add software multiply path, while the hardware accelerator's cycle count is *data-independent*—it always completes in 673 cycles regardless of operand values.

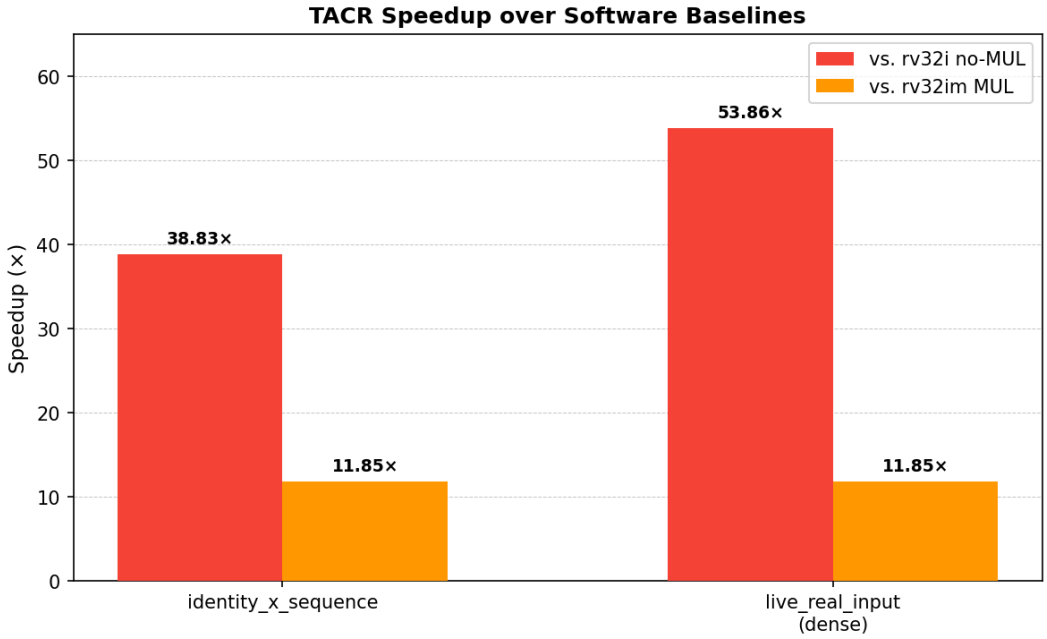


Figure 4.6: Speedup achieved by EdgeMATX across the two benchmark profiles. The MUL-baseline speedup (11.85×) is data-independent, while the no-MUL speedup increases with operand density (38.83× to 53.86×) because the shift-and-add software path requires more instructions for larger values.

### 4.6.5 Why the Acceleration Is Effective

**Against the no-MUL software baseline:** The rv32i software path has no hardware multiply instruction. Each of the 16 inner-product accumulations in the $4 \times 4$ multiply requires a multi-instruction shift-and-add sequence. The hardware accelerator replaces the entire nested loop with a single PCPI transaction, amortizing the fixed interface overhead (memory reads, accelerator start, writeback) across all 16 outputs in one shot.

**Against the MUL-enabled software baseline:** Even with `ENABLE_MUL=1` and `rv32im` firmware, the software path still executes a 3-level nested loop with per-element instruction fetch and dispatch overhead. The accelerator eliminates this loop overhead entirely. The 11.85× residual speedup reflects the loop-control and memory-access instruction overhead that remains in the software path even after multiply cost is reduced.

**Data-independence of the accelerator:** The hardware accelerator's cycle count (673) is invariant to the numerical values of the input matrices. This provides predictable, worst-case-bound execution latency, which is a desirable property for real-time systems.

### 4.6.6 Benchmark Fairness and Standardization

**Unified firmware source.** All three configurations compile the *same* C source file (`firmware_matmul_unified.c`) using preprocessor flags (`MATMUL_MODE_ACCEL` / `MATMUL_MODE_SW`) that select the `run_matmul` implementation at compile time. Entry point, stack setup, `copy_inputs`, and sentinel write are bytecode-identical across all three binaries.

**Identical input data.** Matrix operands `a_init[16]` and `b_init[16]` are compile-time constants embedded in every firmware binary. Every configuration runs against the same bit-for-bit input values; there is no runtime randomness.

**Identical clock and reset conditions.** All testbenches use the same free-running clock and a fixed 8-cycle reset hold before `resetn = 1`. The counter starts at the same architectural moment (CPU out of reset) for every configuration.

**Equal constant-overhead phases.** The boot (Phase 1), input copy (Phase 2), and sentinel write (Phase 4) execute the same instruction sequences and access the same memory addresses in all three configurations. Their contribution cancels out in any speedup ratio computation.

**ISA difference is intentional, not a bias.** The MUL-enabled baseline deliberately uses `rv32im` with firmware compiled for `rv32im`; both the core and the compiler must agree or the multiply instruction is not exercised. This configuration represents the strongest realistic software competitor for the accelerator and is reported separately so its advantage over plain rv32i is also visible.

**Automated, non-manual measurement.** Cycle counts are extracted by parsing the `TB_CYCLES` log line emitted by every testbench; no values are read manually from waveforms. The same automation script processes all three outputs, making the comparison reproducible and auditable.

## 4.7 Consolidated Verification Summary

Table 4.9: Consolidated functional validation status across all verification layers.

| **Flow** | **Script** | **Date** | **Result** |
|---|---|---|---|
| Standalone hardening | `run_midsem_sim` | 2026-03-05 | PASS (19/19) |
| Smoke ASM/C | `run_pcpi_demo` | 2026-03-05 | PASS |
| 8-case regression | `run_pcpi_regression` | 2026-03-05 | PASS (8/8) |
| Handoff test | `run_pcpi_handoff` | 2026-03-05 | PASS |
| Professor cases | `run_pcpi_professor_demo` | 2026-03-05 | PASS (5/5) |
| One-command gate | `run_pcpi_local_check` | 2026-03-05 | PASS |

Table 4.10: Performance summary: speedup across both benchmark profiles.

| **Comparison** | **Speedup — identity case** | **Speedup — dense case** |
|---|---|---|
| Accelerator vs. software (no MUL) | 38.83× | 53.86× |
| Accelerator vs. software (MUL) | 11.85× | 11.85× |

Key observations:

- The hardware accelerator achieves at minimum 11.85× speedup even when compared against a software path equipped with a hardware multiplier.
- Against a plain rv32i core, speedup reaches 38–54× depending on input data density.
- The accelerator cycle count (673 cycles) is invariant to operand values, providing deterministic execution latency.
- A single custom instruction replaces the entire $4 \times 4$ nested-loop execution in the CPU, and all result elements are produced and committed to memory within one PCPI transaction.

## 4.8 Current Limitations and Pending Measurements

The verification evidence at this stage is entirely simulation-based. The following items remain before the project reaches full hardware closure:

- **FPGA deployment:** The design has not yet been mapped to a physical FPGA target (Pynq-Z2). Post-synthesis timing, LUT / DSP48 resource utilization, BRAM usage, and on-board power measurements are not yet available.

- **On-board cycle validation:** Simulation-derived cycle counts will need to be cross-checked against hardware cycle counter measurements (e.g., via PicoRV32 `rdcycle` CSR) once the design is deployed on the Pynq-Z2.
- **Cycle estimator calibration:** The cycle-scaling estimator tool retains an older accelerator anchor of 869 cycles; the latest measured value is 673 cycles. This discrepancy should be corrected before hardware-phase estimation is used.
- **Memory bandwidth characterization:** Current measurements include time for the PCPI wrapper's software-driven memory reads of the input matrices. When these are replaced by BRAM-backed access on the FPGA, the memory-access contribution to the cycle count may change.

These gaps do not affect the correctness evidence; all functional verification layers pass. Closing the hardware measurement gap is the single remaining objective of the next project phase.

# Chapter 5

# Conclusion

The EdgeMATX project has reached a complete and reproducible simulation-stage milestone. The core design—a Q5.10 fixed-point 4×4 systolic array accelerator tightly coupled to a PicoRV32 core via the PCPI custom-instruction interface—has been fully verified through a four-layer simulation campaign: standalone unit tests (19/19 passing), CPU-integration regression (8/8 cases), protocol handoff validation, and quantified cycle-count benchmarking against two software baselines.

The measured performance results establish the value of the hardware approach: the accelerator delivers a 38.83×–53.86× speedup over a plain rv32i software baseline and 11.85× speedup even against a MUL-equipped rv32im core, with a deterministic 673-cycle execution time independent of input data values.

The single remaining objective is hardware closure: deploying the proven simulation contract onto the Pynq-Z2 FPGA platform, measuring on-board cycle counts, and capturing post-synthesis timing, resource utilization, and power data. The simulation-based verification framework, firmware, and automation scripts developed in this phase serve as the direct baseline for board-level validation.

# Bibliography


[1] O. Bagdasar, "Matrix multiplication diagram," Figure 5 in *Matrix Applications in Computer Graphics*. Available on ResearchGate, 2013, original DOI: 10.1007/978-3-319-01751-8_8. [Online]. Available: https://www.researchgate.net/figure/Matrix-multiplication-diagram_fig5_299679931

[2] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for RISC-V," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, 2014. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html

[3] N. P. Nishchay, "Risc-v accelerator architecture visualizer," Interactive web-based architecture simulation, 2026, interactive visualization developed for demonstrating system architecture and accelerator dataflow. [Online]. Available: https://riscv-visualizer-2.vercel.app/

[4] H.-T. Kung and C. E. Leiserson, "Systolic arrays (for VLSI)," in *Sparse Matrix Proceedings 1978*. SIAM, 1979, pp. 256–282.

[5] R. Mattimani, S. V. Siddamal, H. M. Vijay, and S. Shirol, "FPGA implementation of systolic array architecture for matrix multiplication," in *ICT for Intelligent Systems (ICTIS 2024)*, ser. Lecture Notes in Networks and Systems, vol. 1112. Singapore: Springer, 2024.

[6] A. H. M. Shapri, N. Abdul Rahman, S. M. M. S. Zakaria, K. K. Chieh, and S. Saat, "Optimization and analysis of FPGA-based systolic array for matrix multiplication," in *AIP Conference Proceedings*, vol. 2898, no. 1, 2024, p. 030007.

[7] X. Wei, C. Yu, P. Zhang *et al.*, "Highly fault-tolerant systolic-array-based matrix multiplication," *Electronics*, vol. 13, no. 9, p. 1780, 2024.

[8] W. Ye *et al.*, "High-frequency systolic array-based transformer accelerator on field programmable gate arrays," *Electronics*, vol. 12, no. 4, p. 822, 2023.

[9] C. Wolf, "PicoRV32 – A size-optimized RISC-V CPU," GitHub repository, YosysHQ, 2024, accessed 2026. [Online]. Available: https://github.com/YosysHQ/picorv32

[10] K. K. Balasubramanian *et al.*, "Designing RISC-V instruction set extensions for artificial neural networks: An LLVM compiler-driven perspective," *IEEE Access*, 2024.

[11] G. Salinas, G. Sequeira, A. Rodríguez, J. Bispo, and N. Paulino, "Implementation and analysis of custom instructions on RISC-V for edge-AI applications," in *Proceedings of the 14th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies (HEART '24)*. ACM, 2024.

[12] A. Sharma *et al.*, "EXTREM-EDGE – EXtensions to RISC-V for energy-efficient ML inference at the edge of IoT," *Microprocessors and Microsystems*, vol. 94, p. 104620, 2022.

[13] Anonymous, "FPGA-accelerated RISC-V ISA extensions for efficient neural network inference on edge devices," arXiv preprint, 2025.

[14] T. T. Nguyen *et al.*, "Neuromorphic processor-oriented hybrid Q-format multiplication with adaptive quantization for tiny YOLO3," *Neural Computing and Applications*, 2023.

[15] D. Dai *et al.*, "QFX: Trainable fixed-point quantization for deep learning acceleration on FPGAs," arXiv preprint, 2024.

[16] X. Zhang, O. Krestinskaya *et al.*, "Quantized convolutional neural networks: A hardware perspective," *Frontiers in Electronics*, 2025.

[17] E. Tasci, A. Istanbullu *et al.*, "FPGA-QNN: Quantized neural network hardware acceleration on FPGAs," *Applied Sciences*, vol. 15, no. 2, p. 688, 2025.

[18] A. Osman *et al.*, "TinyML: Enabling of inference deep learning models on ultra-low-power IoT edge devices for AI applications," *Micromachines*, vol. 13, no. 6, p. 851, 2022.

[19] S. Somvanshi, "From tiny machine learning to tiny deep learning: A survey," arXiv preprint, 2025.

[20] P. Adithya Krishna *et al.*, "RAMAN: A re-configurable and sparse TinyML accelerator for inference on edge," *IEEE Internet of Things Journal*, 2024.

[21] Ch Madhu Bhushan *et al.*, "Deploying TinyML for energy-efficient object detection and communication in low-power edge AI systems," *Scientific Reports*, 2025.

[22] F. Pazmiño Ortiz *et al.*, "Advancing TinyML in IoT: A holistic system-level perspective for resource-constrained AI," *Future Internet*, vol. 17, no. 6, p. 257, 2025.

[23] M. O. S. Omar, "Risc-v processor and coprocessor signal-level visualizer," Interactive web-based processor signal simulation, 2026, interactive visualization

showing processor execution signals, PCPI handshake, and systolic accelerator behavior. [Online]. Available: https://tinyml-pcpi-visualizer.vercel.app/

[24] J. Bergeron, *Writing Testbenches: Functional Verification of HDL Models*, 2nd ed. Springer Science & Business Media, 2003.

[25] S. Williams, "Icarus Verilog," http://iverilog.icarus.com/, 2024, open-source Verilog simulation and synthesis tool.

[26] A. J. Bybell, *GTKWave 3.3 Wave Analyzer User's Guide*, 2023, open-source waveform viewer. https://gtkwave.sourceforge.net/.